

Certificate of Advanced Studies in AI for Software Engineering

Projektskizze CAS AISE

(umfasst max. 2 Seiten mit Angaben zu untenstehenden inhaltlichen Aspekten der geplanten CAS-Arbeit)

Vorname: **Sinian**

Nachname: **Zhang**

Strasse: **Stockweg 15**

Plz Ort: **5022 Rombach**

Tel.: **+41 79 558 7401**

E-Mail: sinian.zhang@students.fhnw.ch (Privat: zhangsinian@hotmail.com)

Datum: 05.05.2026

Themenbereich und (vorläufiger) Arbeitstitel

Titel: KI PHPUnit Test Generator

Untertitel: Automatische Generierung von PHPUnit-Tests unter der TYPO3-Infrastruktur

Einführung: Die Qualitätssicherung in der Software-Entwicklung basiert auf automatisierten Tests. Aus meiner Erfahrung als TYPO3-Backend-Entwickler fehlen in der Praxis diese jedoch häufig - nicht aus mangelndem Bewusstsein, sondern weil das Schreiben von Tests zeitaufwändig und repetitiv ist. Dieses Projekt untersucht, ob ein LLM bestehenden PHP-Codes lesen und daraus sinnvolle, ausführbare PHPUnit-Tests generieren kann? Im Rahmen des Projektes wird primär nur PHPUnit-Test befasst, um den Projektumfang zu minimieren. Für mich ist das eine super Gelegenheit, intensiv in PHPUnit einzuarbeiten, mein Wissen in diesem Bereich zu vertiefen.

Ausgangslage und Problemstellung

Das manuelle Schreiben von PHPUnit-Tests erfordert tiefes Verständnis des Codes, des TYPO3 Testing Frameworks sowie der zu testenden Abhängigkeiten. (z.B. Repository, DataHandler, Context) Da Entwickler unter Zeitdruck oft nur die Geschäftslogik implementieren, bleibt die Testabdeckung gering.

Mit den modernen PHP-Versionen (8.x) und neue Test-Standards in TYPO3 13/14, nämlich TYPO3 TestingFramework, muss automatische Generierung von PHPUnit-Tests mit KI-Unterstützung zumutbar sein.

Die Problemstellung liegt darin, eine KI so zu instruieren („Prompt Engineering“), dass sie nicht nur validen PHP-Code, sondern framework-konforme, ausführbare und korrekte Test-Mocks für TYPO3-Abhängigkeiten erzeugt.

s. <https://docs.typo3.org/m/typo3/reference-coreapi/main/en-us/Testing/UnitTesting/Introduction.html> -> wenn to unit test, wenn not to unit test.

Das Projekt verfolgt einen zweistufigen Ansatz.

1. Priorisierung: Isolierte Logik & Algorithmen

Der Fokus liegt primär auf Code-Bereichen, die prädestiniert für Unit-Tests sind. Dazu gehören Methoden zur Datenverarbeitung, Datumsberechnungen oder reguläre Ausdrücke. Dabei nutzen wir gezielt Data-Provider, um verschiedene Szenarien effizient durchzuspielen.

2. Sekundärer Bereich: Framework-Integration ("Glue-Code")

Im nächsten Schritt geht auf Code-Bereichen, die vorwiegend als „Klebstoff“ (Glue-Code) zwischen verschiedenen Framework und Funktionalitäten dienen. Der Test mit der Abhängigkeiten wird hier gezielt mit Mocking gearbeitet.

Eventueller Tool und Referenzen, die als Inspirationsquelle dienen und die technische Umsetzung des Projekts unterstützen

<https://github.com/laravel/pao>

Laravel PAO bietet eine für KI-Agenten optimierte Ausgabe (*agent-optimized output*) für PHP-Werkzeuge. Es funktioniert mit jedem PHP-Projekt.

Kernfragen und Überlegungen:

- Kann ein LLM (z.B. Claude API) PHP-Methoden analysieren und korrekte PHPUnit-Tests ableiten?
- Welche Qualität haben die generierten Tests (Korrektheit, Abdeckung, sinnvolle Assertions)?
- Wie lässt sich das Tool in den TYPO3-Entwicklungsworkflow integrieren?

Erste (Ergebnis-)Ziele

Die Ziele der Arbeit stehen noch nicht ganz fest. Sie könnten folgendes erfassen

1. Eine Skript oder ein Kommando, das eine oder eine Menge von PHP-Klassen einliest und via LLM-API automatisch eine PHPUnit-Testklasse mit mindestens 3 sinnvollen Testmethoden pro public-Methode generiert, in einer lokalen Entwicklungsumgebung (DDEV/PHPUnit 13).
2. Vergleichsanalyse zwischen manuell erstellten und KI-generierten Tests.
3. Dokumentation und Ableitung von Empfehlungen, z.B: welche Klassen-Typen sich gut für KI-Testgenerierung eignen und wo manuelle Nacharbeit nötig bleibt.
4. Ein Prompt-Framework integriert in TYPO3, das TYPO3-spezifische Design-Patterns (z.B. Dependency Injection) erkennt und generiert Tests unter Verwendung des TYPO3 Testing Frameworks.

Nutzen

- Primär für TYPO3-Entwickler: schnellere Erhöhung der Code-Coverage in bestehenden Projekten und eine geringere Einstiegshürde für PHPUnit-Testing.
- Eventuell für Agenturen/Unternehmen: Höhere Stabilität und Qualität, schnellere Release-Zyklen (CI/CD) durch bessere Testabdeckung.
- Eventuell für TYPO3-Community: Praktischer Nachweis, dass LLMs für Code-PHPUnit-Test-Generierung unter TYPO3-Infrastruktur gewissermassen produktiv eingesetzt werden können. Veröffentlichung als TYPO3-Extension oder CLI-Tool auf Packagist/GitHub.

Potentielle Betreuung

Fragezeichen, ich bin noch nicht sicher, Empfehlung ist willkommen.